SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA 0603 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO3: *Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)*

Assessment Tool Weightage: 40%

QUESTIONS: 15 **Total Marks:25**

Annexure A

EXPERIMENTS

Code of Conduct:

I Peta Harsha Vardhan (192311039) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Short Answer Test

1. Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

3. Implement the Expectation-Maximization (EM) algorithm for a mixture model and conduct experiments to study its convergence behavior. Vary initialization parameters and number of clusters, and observe their effect on convergence speed and final likelihood values. Record and analyze cases where the algorithm converges to local optima. Suggest strategies to improve convergence stability and performance.

4. Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

5. Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Question 1

Question: Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

Aim

To implement a Naïve Bayes classifier on a real-world dataset, train it with different feature sets, evaluate it using accuracy, precision and recall, and study the effect of the feature-independence assumption and of Laplace smoothing on the results.

Concept Involved

Bayes theorem, Naïve Bayes classifier, conditional independence assumption, Laplace (additive) smoothing, zero-frequency problem, probability estimation.

Definition

Naïve Bayes classifies an example $x = (x_1, \ldots, x_n)$ by choosing the class C that maximises $P(C) \times \prod P(x_i \mid C)$. The "naïve" assumption is that the features are conditionally independent given the class. For categorical features the probability is estimated by counting: $P(x_i = v \mid C) = (\text{count of v in class C} + \alpha) / (N_C + \alpha \times V)$, where V is the number of possible values. With $\alpha = 0$ (no smoothing) a value never seen in a class gets probability 0 and wipes out the whole product; **Laplace smoothing** ($\alpha = 1$) prevents this.

Experimental Setup

Item	Details
Dataset	Wisconsin Breast Cancer Diagnostic dataset (UCI; real clinical data): 569 patients, 30 numeric features, 212 malignant and 357 benign
Task	Binary classification; positive class = malignant
Preprocessing	Each feature converted to 10 equal-frequency bins (fitted on the training fold only), so that probabilities are estimated from counts
Model	Categorical Naïve Bayes (scikit-learn), $\alpha \approx 0$ (1e-10) against $\alpha = 1$ (Laplace)
Evaluation	5-fold stratified cross-validation repeated 5 times (25 test folds); mean accuracy, precision and recall
Feature sets	F1: texture, smoothness, symmetry, fractal dimension (4, weakly correlated, max $\lvert r \rvert = 0.58$) F2: radius, perimeter, area (3, highly correlated, $r \geq 0.987$) F3: all 10 "mean" features F4: all 30 features

Procedure (Step-by-Step)

Step 1: Load the dataset and set malignant = 1 (positive class).

Step 2: Split into training and test folds; discretise features using bins learned from the training fold.

Step 3: Select a feature set (F1–F4) and train Naïve Bayes with $\alpha \approx 0$and with$\alpha = 1$

Step 4: Predict the test fold and compute accuracy, precision and recall; average over the 25 folds.

Step 5: Study correlation between features to judge the independence assumption.

Step 6: Stress test the smoothing: train on only 60 samples with 20 bins, 100 repetitions, and also measure log-loss (quality of the probabilities).

Implementation (key code)

```
kb = KBinsDiscretizer(10, encode="ordinal", strategy="quantile")
Xtr = kb.fit_transform(X_train[:, idx]).astype(int)
Xte = kb.transform(X_test[:, idx]).astype(int)
for alpha in (1e-10, 1.0):
    nb = CategoricalNB(alpha=alpha, min_categories=10).fit(Xtr, y_train)
    pred = nb.predict(Xte)
    print(accuracy_score(y_test, pred), precision_score(y_test, pred),
          recall_score(y_test, pred))
```

Results

Table 1: Performance with different feature sets (cross-validated, %)

Feature set	Smoothing	Accuracy	Precision	Recall
F1: 4 weakly correlated features	No smoothing	78.42	73.59	66.12
	Laplace (α = 1)	78.24	73.32	65.94
F2: 3 highly correlated size features	No smoothing	88.93	91.89	77.44
	Laplace (α = 1)	88.93	91.89	77.44
F3: all 10 mean features	No smoothing	93.04	92.21	88.96
	Laplace (α = 1)	93.18	91.90	89.72
F4: all 30 features	No smoothing	94.45	93.31	91.79
	Laplace (α = 1)	94.38	92.56	92.45

Table 2: Gaussian Naïve Bayes on the raw (undiscretised) features for reference

Feature set	Accuracy	Precision	Recall
F1: 4 weakly correlated features	75.50	71.45	57.37
F2: 3 highly correlated size features	88.54	90.24	77.92
F3: all 10 mean features	91.14	90.26	85.65
F4: all 30 features	93.96	94.67	88.87

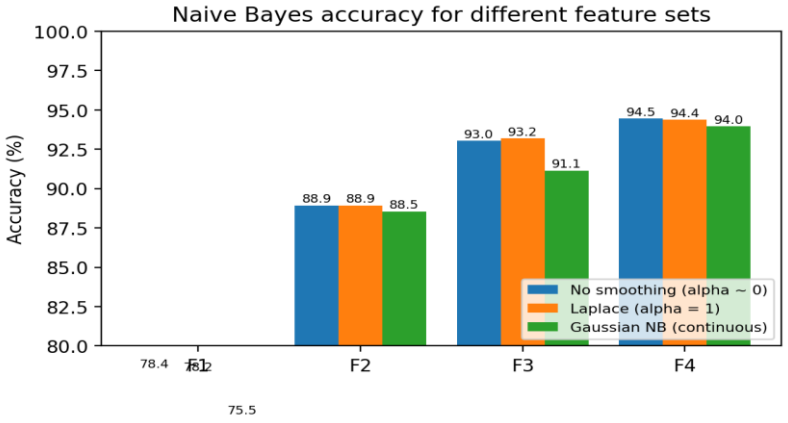


Fig. 1: Accuracy of Naïve Bayes for the four feature sets

Table 3: Smoothing stress test (60 training samples, 20 bins, 10 mean features, 100 repetitions)

Setting	Accuracy	Precision	Recall	Log-loss	Test cases with extreme probability (<0.01 or >0.99)
No smoothing ($\alpha \approx 0$)	90.04	90.56	82.15	2.48	95.8%
$\alpha = 0.5$	90.55	90.26	83.96	–	–
Laplace ($\alpha = 1$)	90.34	90.76	82.76	0.33	70.0%

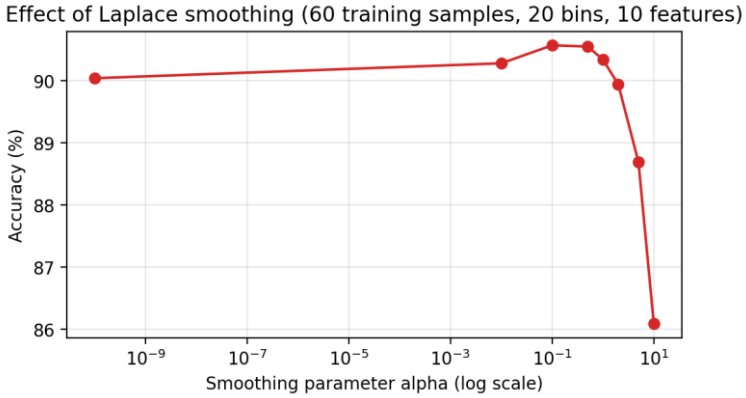


Fig. 2: Accuracy against the smoothing parameter α

Analysis and Interpretation

- **Effect of the feature set.** Accuracy rises from 78.4% with the four weak features (F1) to 94.5% with all 30 features (F4). The three size features alone (F2) give 88.9% accuracy, so a few informative features beat many uninformative ones, but recall for F2 is only 77.4%, so about one malignant case in four is missed.
- **Independence assumption.** Radius, perimeter and area are almost the same measurement (r = 0.998 and 0.987). Naïve Bayes counts this evidence three times, so its probabilities become over-confident, yet its decisions stay fairly accurate because the class is chosen by comparing two scores and not by the exact probability. Using all 30 features still gives the best result even though many are correlated, which shows that Naïve Bayes is robust in classification but poor at giving well-calibrated probabilities.
- **Zero-frequency problem.** In the data, the largest-area bin never occurs in a benign case (0 of 357) and the smallest-area bin never occurs in a malignant case (0 of 212). Without smoothing P(bin | class) = 0 and one such feature makes the entire class score 0, whatever the other features say. Laplace smoothing gives these cases small non-zero probabilities (for example 1/367 = 0.0027 for benign).
- **Effect on cross-validated accuracy.** With enough training data (about 455 samples) smoothing changes accuracy by at most 0.2 points, because zero counts are rare. In the sparse stress test, 99.2% of test cases contain a feature value unseen for one of the classes. Accuracy changes only slightly (90.0% to 90.3%), but log-loss falls from 2.48 to 0.33 and the share of extreme (0 or 1) probabilities falls from 95.8% to 70.0%.
- **Choice of α.** Accuracy is best for α between 0.1 and 0.5 (90.57%–90.55%) and drops to 86.1% at α = 10, because too much smoothing pushes every probability towards uniform and the data are ignored.

Application
Naïve Bayes is used for spam filtering, document classification, medical screening and sentiment analysis, where it is fast, needs little data and works with many features. Smoothing is essential in text, where most words are absent from a given class.

Conclusion
The experiment shows that Naïve Bayes reaches 94.5% accuracy (precision 93.3%, recall 91.8%) with all 30 features. Violating the independence assumption distorts the probability values more than the final decisions. Laplace smoothing has little effect on accuracy when data are plentiful, but it avoids zero probabilities and greatly improves probability estimates (log-loss 2.48 → 0.33) when data are sparse.

Question 2

Question: Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

Aim
To estimate the mean and variance of a distribution using Maximum Likelihood Estimation (MLE) and Bayesian estimation, and to study how the prior and the sample size affect the estimates and their reliability.

Concept Involved
Maximum Likelihood Estimation, Bayesian parameter estimation, conjugate prior (Normal–Inverse-Gamma), posterior distribution, bias and variance of an estimator, credible interval.

Definition
MLE chooses the parameters that maximise the likelihood of the data. For a Gaussian: $\hat{\mu} = (1/n)\ \Sigma x_i$ and $\hat{\sigma}^2 = (1/n)\ \Sigma(x_i - \hat{\mu})^2$ (this variance is biased low by the factor $(n - 1)/n$). **Bayesian estimation** combines a prior with the likelihood to give a posterior. With a Normal–Inverse-Gamma prior $(\mu_0, \kappa_0, \alpha_0, \beta_0)$:

$$\kappa_n = \kappa_0 + n, \quad \mu_n = (\kappa_0\mu_0 + n\,\bar{x}) / \kappa_n, \quad \alpha_n = \alpha_0 + n/2$$

$$\beta_n = \beta_0 + \tfrac{1}{2}\,\Sigma(x_i - \bar{x})^2 + \kappa_0\, n\, (\bar{x} - \mu_0)^2 / (2\kappa_n)$$

Posterior mean of $\mu = \mu_n$; posterior mean of $\sigma^2 = \beta_n / (\alpha_n - 1)$

The weight given to the prior mean is $\kappa_0 / (\kappa_0 + n)$, so the prior matters a lot when n is small and very little when n is large.

Experimental Setup

Item	Details
Dataset	Mean radius of 569 tumours (Wisconsin Breast Cancer dataset). Reference values from all 569 samples: mean = 14.127, variance = 12.397
Sample sizes	n = 3, 5, 10, 20, 50, 100, 300, each drawn at random from the data and repeated 2000 times

Item	Details
Estimators	MLE; Bayesian with three priors
Prior A (weak)	$\mu_0 = 10$, $\kappa_0 = 1$, $\alpha_0 = 2$, prior variance guess 12
Prior B (good)	$\mu_0 = 14$, $\kappa_0 = 5$, $\alpha_0 = 3$, prior variance guess 12
Prior C (wrong, strong)	$\mu_0 = 10$, $\kappa_0 = 10$, $\alpha_0 = 3$, prior variance guess 4
Metrics	Bias and mean squared error (MSE) of the estimated mean and variance against the reference values

Procedure (Step-by-Step)

Step 1: Compute the reference mean and variance from the full data.

Step 2: Draw a random sample of size n and compute the MLE of the mean and variance.

Step 3: Compute the posterior parameters (μ_n, κ_n, α_n, β_n) for each prior and the posterior-mean estimates.

Step 4: Repeat 2000 times for each n and record the bias and MSE of every estimator.

Step 5: Study one sample in detail (n = 5 and n = 100) with 95% intervals.

Implementation (key code)

```
def bayes(s, mu0, k0, a0, v0):
    n, xb = len(s), s.mean(); ss = ((s - xb) ** 2).sum(); b0 = v0 * (a0 - 1)
    kn = k0 + n; mun = (k0 * mu0 + n * xb) / kn; an = a0 + n / 2
    bn = b0 + 0.5 * ss + k0 * n * (xb - mu0) ** 2 / (2 * kn)
    return mun, bn / (an - 1)          # posterior mean of mu and sigma^2
```

Results

Table 1: One sample in detail (reference mean = 14.13)

Estimator	n = 5: mean	n = 5: variance	n = 5: 95% interval	n = 100: mean	n = 100: 95% interval
MLE	13.96	7.96	[10.05, 17.88]	13.67	[13.01, 14.34]
Bayes, weak prior	13.30	10.98	[10.60, 16.00]	13.64	[12.98, 14.29]
Bayes, good prior	13.98	9.75	[12.01, 15.95]	13.69	[13.05, 14.33]
Bayes, wrong prior	11.32	12.01	[9.54, 13.10]	13.34	[12.69, 13.99]

Table 2: Mean squared error of the estimated mean (2000 repetitions)

n	MLE	Weak prior	Good prior	Wrong strong prior
3	4.217	3.447	0.600	10.314
5	2.500	2.173	0.627	7.792
10	1.202	1.122	0.535	4.524
20	0.590	0.567	0.378	2.123
50	0.233	0.231	0.193	0.643
100	0.103	0.102	0.093	0.219
300	0.019	0.019	0.018	0.034

Table 3: Bias of the estimated variance (reference variance = 12.40)

n	MLE	Weak prior	Good prior	Wrong strong prior
3	−3.986	0.629	−0.802	0.481
5	−2.087	0.752	−0.635	2.438
10	−1.080	0.426	−0.594	3.392
20	−0.583	0.203	−0.450	3.051
50	−0.275	0.045	−0.263	1.780
100	−0.098	0.063	−0.104	1.088
300	−0.039	0.015	−0.043	0.395

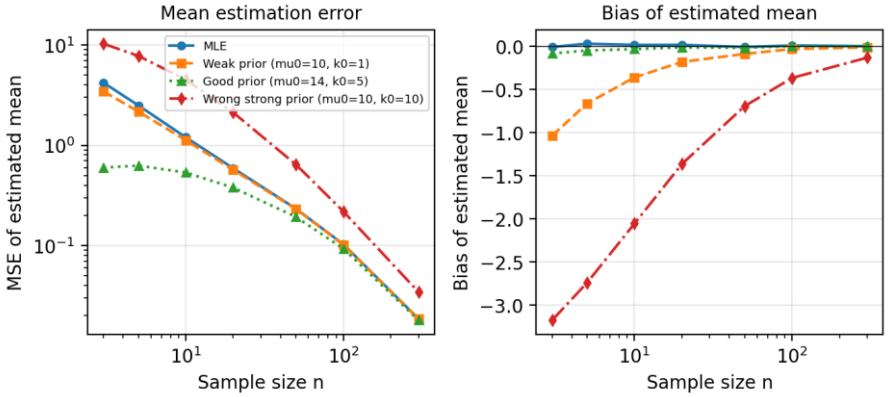


Fig. 1: Error and bias of the estimated mean against sample size

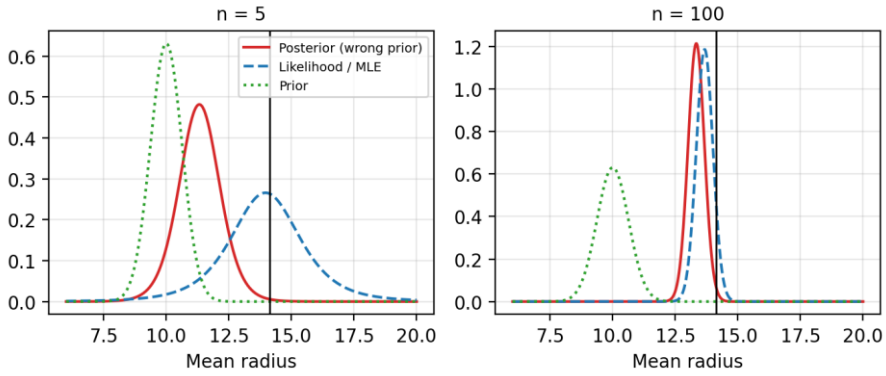


Fig. 2: Prior, likelihood and posterior of the mean (wrong strong prior), n = 5 and n = 100

Analysis and Interpretation

- **Small samples (n = 3–10).** The MLE of the mean is unbiased but very noisy (MSE 4.22 at n = 3), and its variance estimate is biased low (−3.99 at n = 3, close to the theoretical $-\sigma^2/n$). A sensible prior (B) reduces the MSE of the mean to 0.60, about 86% lower, and also reduces the variance error.
- **Wrong prior.** A strong but wrong prior (C) is harmful: its MSE at n = 3 is 10.31, more than twice that of MLE, and its 95% interval for n = 5 [9.54, 13.10] does not contain the true mean 14.13. The posterior is pulled towards the prior (Fig. 2, n = 5).
- **Large samples (n = 100–300).** All estimators converge. At n = 300 the MSE of the mean is 0.019 for MLE and 0.018 for the good prior. The prior's weight $\kappa_0/(\kappa_0 + n)$ for

the wrong prior falls from 67% (n = 5) to 9% (n = 100) and 3% (n = 300), so its bias shrinks from −2.74 to −0.13. The data overwhelm the prior.

- **Variance estimates.** The MLE variance bias decreases as n grows (−3.99 → −0.039). The Bayesian estimate shifts the variance towards the prior value, which helps when the prior is right and hurts when it is wrong (bias +2.44 at n = 5).
- **Reliability.** With few samples, MLE alone is unreliable (its 95% interval for n = 5 is [10.05, 17.88], almost 8 units wide) while the Bayesian interval is narrower, but this is only trustworthy if the prior is justified. With many samples the choice makes no practical difference, and the results are reliable under either method.

Application

MLE is the default in large-data machine learning (logistic regression, Gaussian models). Bayesian estimation is preferred when data are scarce, such as clinical trials, rare-event modelling and A/B tests on small groups, where expert knowledge can be included as a prior.

Conclusion

MLE is simple and unbiased for the mean but unstable for small samples and biased for the variance. Bayesian estimation reduces the error when the prior is reasonable but introduces bias when it is wrong. As the sample size grows, the influence of the prior vanishes and both approaches give the same estimates, so prior assumptions are most important, and most risky, for small samples.

Question 3

Question: Implement the Expectation-Maximization (EM) algorithm for a mixture model and conduct experiments to study its convergence behavior. Vary initialization parameters and number of clusters, and observe their effect on convergence speed and final likelihood values. Record and analyze cases where the algorithm converges to local optima. Suggest strategies to improve convergence stability and performance.

Aim

To implement the Expectation-Maximization (EM) algorithm for a Gaussian mixture model and to study how initialisation and the number of clusters affect convergence speed, final likelihood and local optima.

Concept Involved

EM algorithm, Gaussian mixture model (GMM), latent variables, responsibilities, log-likelihood, local optima, model selection (BIC).

Definition

EM finds maximum-likelihood parameters of a model with hidden variables by repeating two steps. For a mixture of K Gaussians with weights π_k, means μ_k and covariances Σ_k:

E-step: $\gamma_{ik} = \pi_k \, N(x_i \mid \mu_k, \Sigma_k) \,/\, \Sigma_j \, \pi_j \, N(x_i \mid \mu_j, \Sigma_j)$ (responsibility of component k for point i)

M-step: $N_k = \Sigma_i \, \gamma_{ik}$, $\pi_k = N_k / N$, $\mu_k = \Sigma_i \, \gamma_{ik} \, x_i / N_k$, $\Sigma_k = \Sigma_i \, \gamma_{ik} \, (x_i - \mu_k)(x_i - \mu_k)^T / N_k$

The log-likelihood never decreases from one iteration to the next, but EM only reaches a local maximum, which depends on the starting values.

Experimental Setup

Item	Details
Dataset	Iris (real data): 150 flowers, 4 features (sepal and petal length and width); class labels are not used for training
Algorithm	EM for a GMM written from scratch in NumPy; full covariance matrices with a small regularisation term (1e-4 × identity); stop when the change in log-likelihood is below 1e-6 (maximum 500 iterations)
Experiment A	K = 3, four initialisations, 100 runs each: (1) random data points as means, (2) "close" start (all means near the overall mean), (3) k-means++ seeding, (4) k-means result
Experiment B	K = 1 to 6 with 20 k-means++ starts each; best run kept; log-likelihood, parameters and BIC recorded
Local optimum	A run is counted as a local optimum if its final log-likelihood is more than 0.5 below the best value found (–180.19)

Procedure (Step-by-Step)

Step 1: Initialise μ_k, Σ_k and π_k (strategy being tested).

Step 2: E-step: compute the responsibilities and the log-likelihood.

Step 3: M-step: update the weights, means and covariances.

Step 4: Stop when the log-likelihood change is below 1e-6; record iterations and final value.

Step 5: Repeat for all initialisations (Experiment A) and for K = 1–6 (Experiment B); compare.

Implementation (key code)

```
for it in range(max_iter):
    L = np.column_stack([np.log(pi[k]) + mvn.logpdf(X, mu[k], S[k])
                         for k in range(K)])
    lse = logsumexp(L, axis=1); ll = lse.sum()
    R = np.exp(L - lse[:, None])                    # E-step
    if abs(ll - prev) < tol: break
    Nk = R.sum(0); pi = Nk / N
    mu = (R.T @ X) / Nk[:, None]                    # M-step
    for k in range(K):
        dx = X - mu[k]
        S[k] = (R[:, k, None] * dx).T @ dx / Nk[k] + reg * np.eye(d)
    prev = ll
```

Results

Table 1: Effect of initialisation, K = 3 (100 runs each)

Initialisation	Mean iterations	Iteration range	Best log-lik.	Mean log-lik.	Worst log-lik.	Runs reaching best	Runs in local optima
Random data points	100.6	17–252	−180.19	−192.12	−280.94	9%	91%
Close start	144.4	60–242	−180.19	−194.57	−282.52	2%	98%
k-means++ seeding	101.0	10–252	−180.19	−189.07	−267.77	14%	86%
k-means result	32.6	32–33	−180.19	−180.19	−180.19	100%	0%

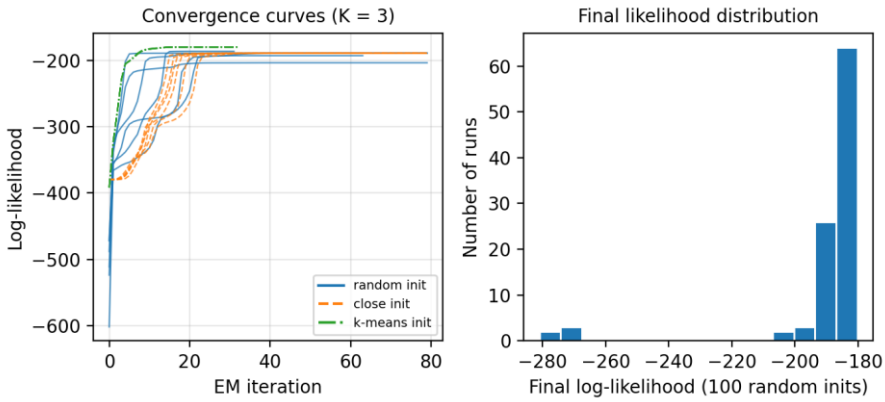


Fig. 1: Convergence curves (left) and distribution of final log-likelihood over 100 random starts (right)

Final log-likelihood values reached from random starts: −267.8 (3 runs), −193.2 (3 runs), −193.1 (9 runs), −189.5 (13 runs), −186.6 (53 runs), −180.2 (9 runs), plus several rare poorer values. Only 9 of 100 random starts reached the best value (–180.2).

Table 2: Effect of the number of clusters K (best of 20 runs)

K	Mean iterations	Log-likelihood	Free parameters	BIC (lower is better)
1	3.0	−379.92	14	830.0
2	25.3	−214.36	29	574.0
3	111.6	−180.19	44	580.8
4	115.1	−137.12	59	569.9
5	88.7	−142.85	74	656.5
6	96.0	−123.36	89	692.7

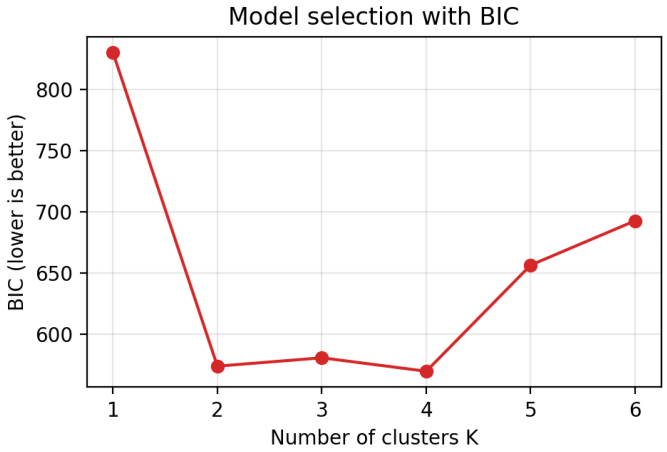


Fig. 2: BIC for different numbers of clusters

Analysis and Interpretation

- **Initialisation and speed.** Starting from the k-means solution converges fastest (about 33 iterations) against about 101 for random points and 144 for the close start. A close start is slowest because the components must first separate from each other.
- **Local optima.** With random points, 91% of runs ended in a local optimum, and with the close start 98%. The worst runs ended at a log-likelihood of −281, far below the best

−180. Local optima of this kind usually correspond to two components sharing one real cluster while another cluster is split or merged. About half of the random runs (53) converged to the same poorer solution (–186.6). k-means initialisation reached the best value in all 100 runs.

- **Number of clusters.** The best log-likelihood rises as K grows (−380 for K = 1 to −137 for K = 4) because more components fit the data better. The K = 5 value (−142.9) is lower than the K = 4 value, which is impossible for the true maximum, so even 20 restarts were not enough for K = 5; the more components, the more local optima. Convergence needs more iterations for larger K (3 iterations for K = 1 against about 90–115 for $K \geq 3$).
- **Model selection.** BIC is lowest at K = 4 (569.9), narrowly ahead of K = 2 (574.0) and K = 3 (580.8). K = 2 separates setosa from the other two species, which overlap. With K = 3 (k-means start), 96.7% of flowers are in a cluster that matches their species.

Strategies to Improve Stability

- **Better initialisation:** use k-means or k-means++ instead of purely random means (k-means gave 100% success with a 3 times speed-up).
- **Multiple restarts:** run EM many times and keep the solution with the highest likelihood (at least 20–100 restarts for larger K).
- **Covariance regularisation:** add a small term to the diagonal to avoid singular covariances when a component collapses on a few points.
- **Model selection:** choose K with BIC or AIC and not by likelihood alone, as likelihood always increases with K.
- **Other measures:** a looser tolerance with a maximum iteration count, deterministic annealing, or merging and splitting components.

Application

EM for mixture models is used in customer segmentation, image and speech modelling, anomaly detection and soft clustering, where points can belong partly to several groups.

Conclusion

EM always improves the likelihood but converges to a local optimum that depends on initialisation: 91% of random starts and 98% of close starts were stuck in poorer solutions for K = 3, while the k-means start reached the best value (–180.19) every time in 33 iterations. Combining good initialisation, several restarts, regularisation and BIC makes EM stable and reliable.

Question 4

Question: Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

Aim

To construct a Bayesian Belief Network for a medical-diagnosis domain, perform exact inference under different evidence conditions, modify the conditional probabilities, and analyse dependencies, robustness and the handling of uncertainty.

Concept Involved

Bayesian Belief Network, conditional probability table (CPT), joint distribution, exact inference by enumeration, conditional independence, explaining away, sensitivity analysis.

Definition

A Bayesian Belief Network is a directed acyclic graph in which each node is a random variable with a CPT given its parents. The joint distribution is the product of the local probabilities: $P(x_1, \ldots, x_n) = \prod P(x_i \mid \text{parents}(x_i))$. Inference computes P(query | evidence) = Σ joint (matching evidence and query) / Σ joint (matching evidence).

Experimental Setup

Domain: lung-cancer diagnosis with five binary variables: Pollution (high / low) and Smoker (yes / no) are causes of Cancer, and Cancer causes a positive X-ray and Dyspnoea (breathlessness). The 32 combinations of the five variables were enumerated in Python and the probabilities computed exactly.

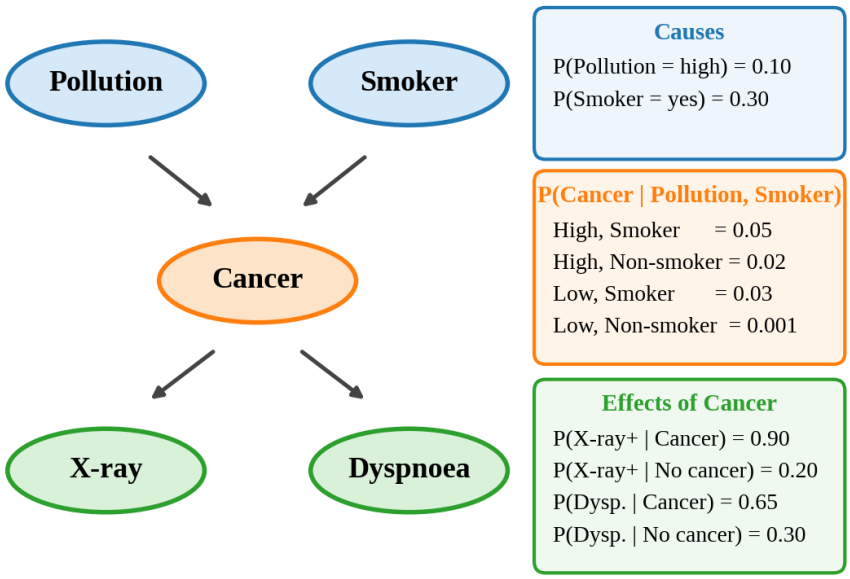


Fig. 1: Bayesian Belief Network with its conditional probability tables

Procedure (Step-by-Step)

Step 1: Define the structure and CPTs (Fig. 1).

Step 2: Compute P(Cancer | evidence) for eleven evidence conditions (Experiment 1).

Step 3: Test the dependencies, including explaining away (Experiment 2).

Step 4: Change individual CPT values and record the posterior (Experiment 3).

Step 5: Perturb every CPT entry randomly by ±20% (2000 times) to measure robustness (Experiment 4).

Implementation (key code)

```
def joint(P, pol, smk, can, xr, dy):       # product of local CPTs
    bern = lambda p, v: p if v else 1 - p
    p = bern(P["pH"], pol) * bern(P["pS"], smk)
    p *= bern(cpt_cancer[(pol, smk)], can)
    p *= bern(P["xC"] if can else P["xN"], xr)
    p *= bern(P["dC"] if can else P["dN"], dy)
    return p
posterior = sum of joint over states matching evidence and Cancer = yes,
          divided by the sum over all states matching the evidence
```

Results

Table 1 (Experiment 1): Posterior probability of cancer under different evidence

Evidence	P(Cancer | evidence)
No evidence	0.0116
Smoker = yes	0.0320
High pollution	0.0290
X-ray positive	0.0503
Dyspnoea = yes	0.0249
X-ray + Dyspnoea	0.1029
X-ray + Smoker	0.1295
X-ray + Dysp. + Smoker	0.2437
X-ray + Dysp. + Smoker + High poll.	0.3391
X-ray negative	0.0015
X-ray neg. + Dysp. no	0.0007

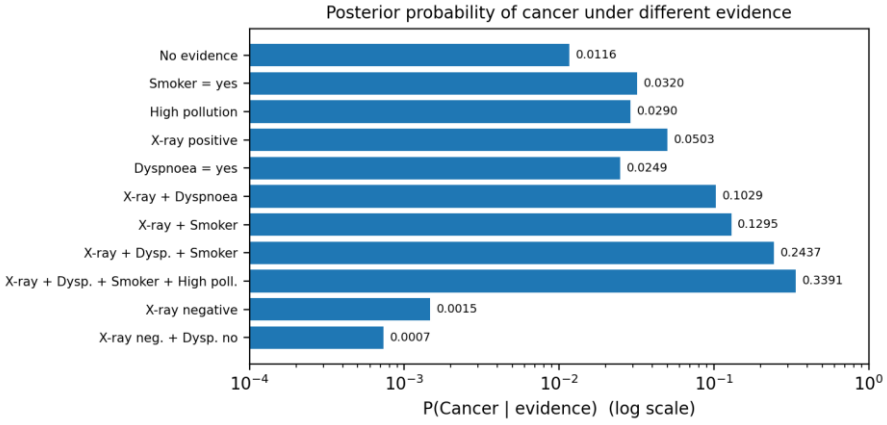


Fig. 2: Posterior probability of cancer under different evidence (log scale)

Table 2 (Experiment 2): Dependencies and explaining away

Quantity	Value
P(Smoker)	0.300
P(Smoker | Cancer)	0.825
P(Smoker | Cancer, High poll.)	0.517
P(Smoker | Cancer, Low poll.)	0.928

Quantity	Value
P(High poll.)	0.100
P(High poll. | Cancer)	0.249
P(High poll. | Cancer, Smoker)	0.156
P(High poll. | Cancer, Non-smoker)	0.690

Table 3 (Experiment 3): Effect of modifying the CPTs

Modification	P(C | X-ray+)	P(C | X-ray+, Dysp.)	P(C | X-ray+, Dysp., Smoker)
Original	0.0503	0.1029	0.2437
X-ray sensitivity 0.90 -> 0.70	0.0396	0.0819	0.2004
X-ray sensitivity 0.90 -> 0.99	0.0550	0.1121	0.2617
X-ray false-positive 0.20 -> 0.05	0.1748	0.3146	0.5632
X-ray false-positive 0.20 -> 0.40	0.0258	0.0543	0.1388
P(Smoker) 0.30 -> 0.60	0.0855	0.1685	0.2437
P(Cancer|Smoker, Low) 0.03 -> 0.10	0.1241	0.2349	0.5058
Dyspnoea given cancer 0.65 -> 0.90	0.0503	0.1371	0.3086
Pollution high 0.10 -> 0.40	0.0739	0.1474	0.2780

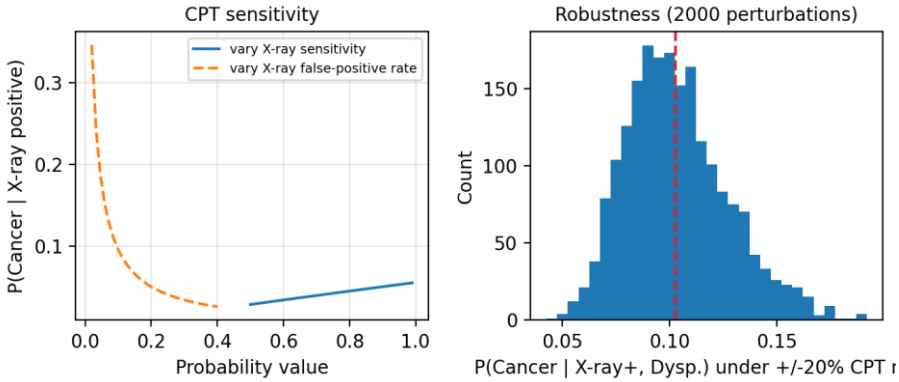


Fig. 3: Sensitivity to X-ray CPT values (left) and robustness to ±20% noise in all CPTs (right)

Table 4 (Experiment 4): Robustness to ±20% random change of every CPT entry (2000 trials)

Query	Original	Mean	5th–95th percentile
P(Cancer | X-ray+, Dysp.)	0.1029	0.1033	0.069 – 0.147
P(Cancer | X-ray+, Dysp., Smoker)	0.2437	0.2433	0.173 – 0.325

Analysis and Interpretation

- **Evidence accumulation.** The prior probability of cancer is only 1.16%. A positive X-ray raises it to 5.0%, adding breathlessness gives 10.3%, being a smoker 24.4%, and living in a high-pollution area 33.9%. A negative X-ray lowers it to 0.15%. Each piece of evidence moves the belief in the expected direction.
- **Why a positive X-ray gives only about 5%.** The disease is rare, so most positive X-rays come from healthy people (false-positive rate 20%). The prior (base rate) dominates the result.

- **Conditional independence.** X-ray and Dyspnoea are independent given Cancer (difference in probability about 10^{-16}) but dependent when Cancer is unknown (difference 0.0093): a positive X-ray raises belief in cancer and so makes breathlessness more likely.
- **Explaining away.** Smoking and pollution are independent causes, but once cancer is known they compete: P(Smoker | Cancer) = 0.825 falls to 0.517 if the area is also known to be highly polluted. Similarly P(High pollution | Cancer) is 0.249, 0.156 if the patient smokes and 0.690 if not.
- **Effect of changing CPTs.** The posterior is most sensitive to the X-ray false-positive rate: reducing it from 0.20 to 0.05 raises P(Cancer | X-ray+) from 0.050 to 0.175, whereas changing the sensitivity from 0.70 to 0.99 only moves it from 0.040 to 0.055. Changing P(Cancer | Smoker, Low pollution) from 0.03 to 0.10 doubles the posterior with full evidence (0.244 → 0.506). Doubling P(Smoker) has no effect once smoking is observed, as it only changes the prior.
- **Robustness.** With ±20% random noise in all CPTs, P(Cancer | X-ray+, Dysp.) stays centred on 0.103 with 90% of values between 0.069 and 0.147. The numbers change by up to about 40%, but the ranking of the evidence conditions and the conclusion (low to moderate risk) do not.
- **Handling uncertainty.** The network never gives a yes or no answer but a probability that is updated when new evidence arrives, combining prior knowledge, causal structure and noisy test results. The sensitivity analysis shows which probabilities (here the false-positive rate) must be estimated carefully.

Application
Bayesian networks are used in medical diagnosis, fault diagnosis, risk assessment, spam and fraud detection and decision support systems, where causes and effects are uncertain.

Conclusion
The Bayesian Belief Network represents the lung-cancer domain compactly and gives exact posterior probabilities for any evidence. The experiments show how evidence changes belief (1.16% → 33.9%), how dependencies such as conditional independence and explaining away arise from the structure, how sensitive the result is to the false-positive rate, and that the conclusions remain stable under ±20% error in the CPTs.

Question 5

Question: Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

Aim
To implement the Candidate Elimination concept-learning algorithm and to study how the size of the hypothesis space (number of attributes) and the number of training examples affect

sample complexity, convergence and generalisation, and to interpret the results with the mistake bound model and the finite / infinite hypothesis space theory.

Concept Involved

Concept learning, version space, Candidate Elimination (S and G boundaries), hypothesis space size, sample complexity, VC dimension, finite and infinite hypothesis spaces, mistake bound (Find-S, Halving algorithm).

Definition

The **version space** is the set of all hypotheses consistent with the training examples. **Candidate Elimination** keeps only its boundaries: S (most specific) and G (most general); positive examples generalise S and prune G, and negative examples specialise G and prune S, until S = G. For a conjunction of n Boolean attributes (each attribute may be 0, 1 or "?") the hypothesis space has $|H| = 3^n + 1$ members. Key bounds:

Sample complexity (finite H, consistent learner): $m \geq (1/\varepsilon)\ [\ \ln|H| + \ln(1/\delta)\ ]$

Mistake bounds: Find-S for conjunctions $\leq n + 1$; Halving algorithm $\leq \log_2|H|$

Infinite H (VC dimension d): $m \geq (1/\varepsilon)\ [\ 4\log_2(2/\delta) + 8\,d\log_2(13/\varepsilon)\ ]$

Experimental Setup

Item	Details
Task	Learn a hidden conjunctive concept over n Boolean attributes (similar to the EnjoySport problem); noise-free examples drawn uniformly at random
Target concept	Random conjunction with k = n/2 specified attributes (at least 2); for n = 8, k = 4
Algorithms	Candidate Elimination, Find-S and the Halving algorithm, all written from scratch in Python
Experiment A	Hypothesis space size and theoretical bounds for n = 2 to 8 (ε = 0.1, δ = 0.05)
Experiment B	Number of examples until the version space collapses to one hypothesis (S = G); n = 3 to 8; 200 trials each
Experiment C	Test accuracy (300 test instances) of the learned S hypothesis and the share of test instances still uncertain, against the number of training examples m (n = 4, 6, 8; 200 trials)
Experiment D	Online mistakes of Find-S and Halving over 300 examples (n = 3 to 7; 60 trials)
Experiment E	Infinite hypothesis space: learning an interval [a, b] on the real line (VC dimension 2), 500 trials

Procedure (Step-by-Step)

Step 1: Initialise S = {∅} and G = { ⟨?, ?, …, ?⟩ }.

Step 2: For a positive example, remove from G the hypotheses that do not cover it and minimally generalise S to cover it.

Step 3: For a negative example, remove from S the hypotheses that cover it and minimally specialise G to exclude it.

Step 4: Repeat for each training example; stop when S = G (the concept is learned).

Step 5: Repeat for different n and m, and record convergence, accuracy and mistakes.

Implementation (key code)

```
def step(S, G, x, positive, n):
    if positive:
        G = [g for g in G if match(g, x)]          # prune G
        S = [generalise(s, x) for s in S]          # minimal generalisation
    else:
        S = [s for s in S if not match(s, x)]      # prune S
        G = specialise_all(G, x, S)                # minimal specialisation
    return S, G
```

Results

Table 1 (Experiment A): Hypothesis space size and bounds

Attributes n	$\|H\| = 3^n + 1$	$\log_2\|H\|$ (Halving bound)	n + 1 (Find-S bound)	PAC sample bound m ($\varepsilon = 0.1$, $\delta = 0.05$)
2	10	3.32	3	53
3	28	4.81	4	64
4	82	6.36	5	75
5	244	7.93	6	85
6	730	9.51	7	96
7	2188	11.10	8	107
8	6562	12.68	9	118

Table 2 (Experiment B): Examples needed for Candidate Elimination to converge (200 trials)

n	Target literals k	Mean examples	Median	90th percentile	Largest |G| seen
3	2	12.1	10	20	6
4	2	15.8	13	26	12
5	2	16.0	14	26	20
6	3	33.2	29	57	48
7	3	38.1	33	70	85
8	4	78.0	72	130	219

Table 3 (Experiment C): Generalisation against the number of training examples

n	Measure	m = 2	m = 5	m = 10	m = 20	m = 50	m = 100	m = 200
4	Accuracy (%)	78.5	83.0	89.6	97.0	99.9	100.0	100.0
	Uncertain (%)	82.0	48.2	19.9	3.6	0.1	0.0	0.0
6	Accuracy (%)	87.8	88.5	90.2	93.7	98.9	99.9	100.0
	Uncertain (%)	94.2	76.3	51.4	19.3	1.4	0.1	0.0
8	Accuracy (%)	93.8	93.9	94.2	94.9	97.1	99.2	99.9
	Uncertain (%)	98.6	91.2	76.7	47.3	9.5	1.1	0.1

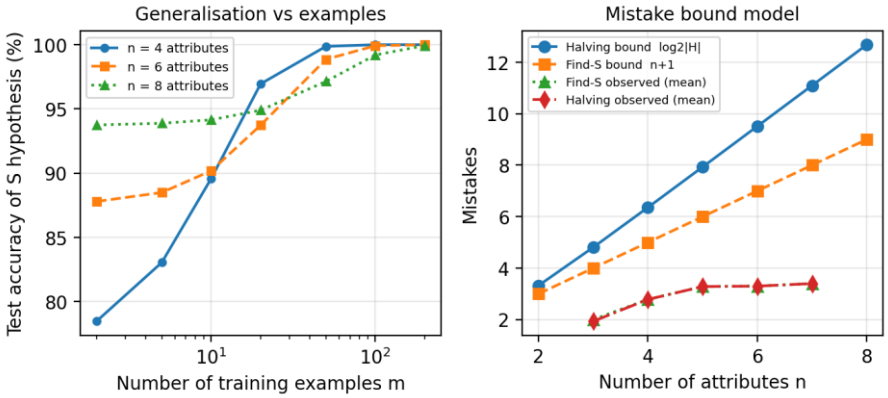


Fig. 1: Generalisation against training examples (left) and mistake bounds against observed mistakes (right)

Table 4 (Experiment D): Online mistakes (mean over 60 trials, maximum in brackets)

n	Find-S mistakes	Halving mistakes	Find-S bound n + 1	Halving bound $\log_2\|H\|$
3	2.00 (2)	1.93 (2)	4	4.81
4	2.78 (3)	2.78 (3)	5	6.36
5	3.28 (4)	3.28 (4)	6	7.93
6	3.30 (4)	3.30 (4)	7	9.51
7	3.40 (5)	3.40 (5)	8	11.10

Table 5 (Experiment E): Infinite hypothesis space – error of the learned interval

Training examples m	2	5	10	20	50	100	200	500
Mean error (%)	29.40	20.49	14.67	8.35	3.82	1.94	1.01	0.38

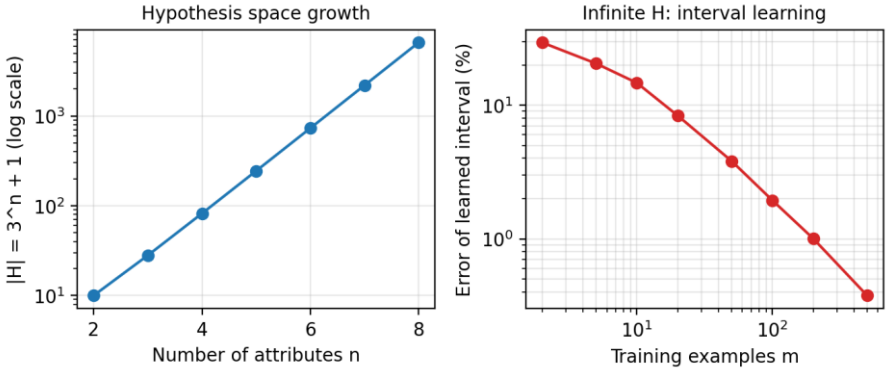


Fig. 2: Growth of the hypothesis space (left) and learning an interval, an infinite space of finite VC dimension (right)

Table 6: Sample bounds from the VC dimension (ε = 0.1, δ = 0.05)

Hypothesis class	VC dimension	Sufficient examples m
Conjunction of n=6 literals (finite, |H|=730)	6	3584
Intervals on a line	2	1337
Linear separators in 2-D	3	1899
Axis-parallel rectangles in 2-D	4	2461

Analysis and Interpretation

- **Hypothesis space size and sample complexity.** $|H|$ grows exponentially, from 28 for n = 3 to 6,562 for n = 8, and the number of examples needed for convergence grows with it, from about 12 to 78 (mean). Convergence (S = G) needs informative negative examples that differ from the target in exactly one attribute, and these are rare in random data, so the count grows faster than $\ln|H| = n \ln 3$. The PAC bound (64 to 118 examples for n = 3 to 8) only guarantees error below 10%, which is much easier: at m = 100 the test accuracy for n = 8 is already 99.2%.
- **Cost of the G boundary.** The G boundary can grow quickly: the largest G seen was 6 for n = 3 and 219 for n = 8, so the computational cost of Candidate Elimination rises sharply with the number of attributes.
- **Generalisation.** Test accuracy rises with m. For n = 4 it is 97.0% at m = 20 and 100% at m = 100; for n = 8 it needs about 100–200 examples to reach 99–100%. Large n looks accurate at small m only because a conjunction with 4 literals is true for just 6.25% of instances: the learner predicts "negative" and is right 93.75% of the time (the value at m = 2). The "uncertain" row shows the true state: 98.6% of test instances are still undecided at m = 2 for n = 8, falling to 1.1% at m = 100.
- **Mistake bound model.** Observed online mistakes (2.0 to 3.4) are well below both bounds ($n + 1 = 4$ to 8; $\log_2|H| = 4.8$ to 11.1), because the examples are random and the bounds are for the worst case. The Halving bound grows only logarithmically in $|H|$, which is linear in n, so learning stays feasible as the space grows exponentially. Find-S and Halving behaved almost identically on these concepts.
- **Finite versus infinite hypothesis spaces.** Intervals on a line form an infinite space, but their VC dimension is 2, so the error falls steadily (29.4% with 2 examples to 0.38% with 500, roughly proportional to 1/m). A finite space has a bound based on $\ln|H|$ and an infinite space of finite VC dimension has a bound based on d, so both are learnable. If the hypothesis space contains every possible concept (no bias), the version space never shrinks, so the learner can only memorise and cannot generalise.

Application

Concept learning and version-space ideas underlie rule learning, expert-system knowledge acquisition and feature-based classifiers. The sample-complexity and mistake-bound results guide the choice of model class and the amount of data needed.

Conclusion

The experiments confirm that larger hypothesis spaces need more examples (about 12 for n = 3 to 78 for n = 8 before Candidate Elimination converges) and that Candidate Elimination becomes more expensive as G grows. Observed mistakes stay far below the Find-S $(n + 1)$ and Halving $(\log_2|H|)$ bounds, and learning is possible in both finite spaces and infinite spaces of finite VC dimension, where it is the VC dimension, not the number of hypotheses, that controls the sample size.